

Chapter 8 _ Dataset engineering

Quality of the model depend strongly on the training data.

Dataset Engineering is to create a dataset that allows you to train the best model, identically within your allocated budget.

- **Data-Centric AI** tries to improve AI performance by enhancing the data. This involves developing new data processing techniques and creating high-quality datasets that allow better models to be trained with fewer sources.
- **Model-Centric AI** tries to improve AI performance by enhancing the models themselves. This involves designing new architectures, increasing the sizes of the models, or developing new training technique

Data Curation

It is all about figuring out what data we actually need and getting our hand on its.

- **Data Quality:** The most important thing. A small amount of high-quality data can outperform a much larger pile of noisy data. It spans things like relevance to the task, correctness, consistency of labeling/formatting, and the absence of the toxic or harmful content.
- **Data Coverage:** dataset has to represent the full range of situations the model will face in production (i.e., different topics, languages, difficulty levels, edge cases, and user intents). and the dataset can still be huge and have poor coverage if it is lopsided/unbalance (i.e., English is massively overrepresented in the dataset, explaining why LLMs are so proficient in English)
- **Data Quantity:** the quantity of the data depends heavily on the adaptation technique (i.e., full pre-training needs vastly more than instruction-tuning, which needs more than a light fine-tune or a few-shot prompt)
- **Data Acquisition and Annotation:** aka where the data physically comes from? like public datasets, web scraping, purchasing data, user-generated data, or manual/human annotation. and the annotation covers labeling guidelines, inter-annotator agreement, quality control on human labelers, and the annotation is expensive, slow and often the bottleneck in a data pipeline.

Data Augmentation and Synthesis

more of creating data than collecting

- **Synthesis:** so if the real data is scarce or expensive (especially for niche domains or safety-relevant edge cases), privacy concerns favor synthetic substitutes, and we can deliberately generate examples to fill coverage gaps that we face above
 - **Traditional Data Synthesis Techniques:** ruled-base generation, perturbation/data augmentation (paraphrasing, back-translation, noise injection),

template-based generation, and simulation.

- **AI Powered Data Synthesis:** using LLMs themselves to generate training data (i.e., prompting a strong model to produce instruction-response pairs, or to generate variations of existing examples). and it is surprisingly effective. It is far cheaper and faster than human annotation and human-written data
 - Pipeline
 - **Seed examples:** start with a small set of real, high-quality data = investment
 - **Prompt the teacher model:** ask the model to generate new example like new questions, new instruction-response pairs, paraphrases of existing prompts, or entirely new scenarios covering identified coverage gaps (e.g., generate me 50 customer support question about billing dispute in a frustrated tone)
 - **Diversity control:** vary the prompt template/instructions, sample at higher temperature (boost creativity), explicitly ask for validation along axes we care about like topic, difficulty, tone, length and also try to use different seed examples for different batches.
 - **Filtering/Verification:** one thing to keep in mind is that the synthetic data is noisier than it looks. So for the filter, we can use another model (or the same model) scores the outputs for quality/correctness. and it catches malformed or off-template outputs, deduplication, and remove the repetition.
 - **Mix with the real data: and also remember** that the synthetic data is rarely used alone, it is typically blended with real examples, since pure synthetic training can amplify the teacher model's own biases and blind spots
- **Model Distillation:** using a teacher model's outputs to train a smaller student model. This is cited as a powerful case of AI-powered synthesis. So instead of manually labelling data, we let a capable model generate the labels/responses, then train a cheaper model to imitate it.
 - **Response-based distillation:** we run the target input through the teacher model, collect its outputs, and fine-tune the student on those pair (input, teacher-output).
 - but there is a practical concerns regarding this method flagging the licensing (as some model providers restrict using their outputs to train competing models, this has been a live legal/policy issue), quality bottleneck (student typically cannot exceed teacher performance on the distilled task), and we still need the same filtering/verification need as synthesis method.

Data Processing

Pipeline to turn the data into something trainable:

- **Inspect the data:** ACTUALLY looking at your data before training on it. Greg Brockman state that manual inspection of the data has probably the highest value-to-prestige ratio of any activity in machine learning. aka eyeballing the distribution, laser focus and try to spot the anomalies, and build the intuition for what is in the data
- **Deduplicate the data:** we have to remove the duplicate data since it is a major cause that introduce bias and skewed model's training. **Deduplication technique:** Exact matching, n-gram overlap, and semantic similarity.
 - **Exact matching:** catch byte-for-byte duplicates, it might miss two documents that differ by a single whitespace character or a reordered sentence.
 - **N-gram overlap (aka Near Duplication Detection):** instead of hashing the whole document, we break it into overlapping n-grams (e.g., sequence of 5-13 words) and compare the sets of n-grams between documents. common implementation like Jaccard Similarity, MinHash + Locality-Sensitive Hashing (LSH)
 - **Semantic Similarity:** catches the duplication that do not share much surface text but say the same thing (like paraphrase, translations, or reworded versions). doing so by using the embedding (embedding each document with a sentence/document embedding model) then run the approximate nearest-neighbor search (**FAISS**, **ScaNN**,...) to find embeddings that are very close in vector space. and last but not least, we flag the pairs above similarity threshold as duplication.
- **Clean and Filtered Data:** removing extraneous formatting like HTML tags, filtering out toxic or non-compliant content, and removing low-quality examples.
- **Format Data:** the final step is matching your data to the exact chat template or input format the model expects for fine-tuning.